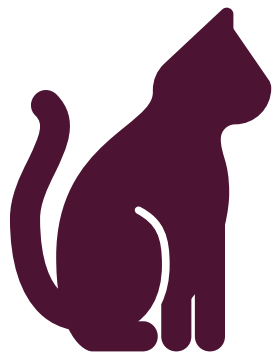


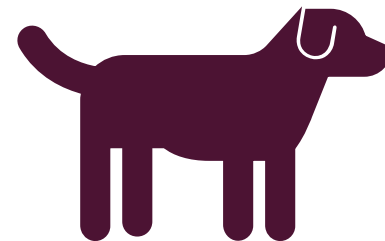
ANY INSIGHTS FROM THIS CODE?

```
class Dog {  
    void sleep() {  
        System.out.println("Dog sleeps");  
    }  
  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat {  
    void sleep() {  
        System.out.println("Cat sleeps");  
    }  
  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}  
  
class Cow {  
    void sleep() {  
        System.out.println("Cow sleeps");  
    }  
  
    void sound() {  
        System.out.println("Cow moos");  
    }  
}
```

THE INSIGHTS



Every animal sleeps or breathes in
a similar way



But every animal makes a different
sound

WHAT SHOULD WE DO?



Should we write the same `sleep()` method again and again in every class?



Can we create one common rule for all animals where:

`sleep()` is already defined
`sound()` must be implemented differently by each animal



I HAVE A GUESS THAT
PROBABLY YOU MAYBE
THINKING....

HIERARCHICAL INHERITANCE ALONE CAN'T SOLVE THIS



We can define `sound()` separately in each child class like Dog, Cat, and Cow



But then how can we make sure every animal class must have a `sound()` method?



What if a programmer creates a new animal class and forgets to write `sound()`?



So how can we force all animal classes to implement `sound()` while still keeping common methods like `sleep()` in one place?



ABSTRACTION

THE PROBLEM

- `sleep()` is the same for all animals
→ can be defined once
- `sound()` is different for each animal → must be defined in each child class
- How do we **force** every animal class to have a `sound()` method?



ABSTRACTION IS THE SOLUTION

It allows us to **define common behavior** in a parent class

It allows us to **force child classes** to implement methods that are different

In Java, abstraction is achieved using **abstract classes** or **interfaces**

EXAMPLE IDEA

- `sleep()` is **already implemented**
- `sound()` is **abstract** → every child class must define it

```
abstract class Animal {  
    void sleep() {  
        System.out.println("Animal sleeps");  
    }  
  
    abstract void sound(); // Must be defined by every  
    animal  
}
```

THIS IS HOW ABSTRACTION SOLVES THIS ISSUE

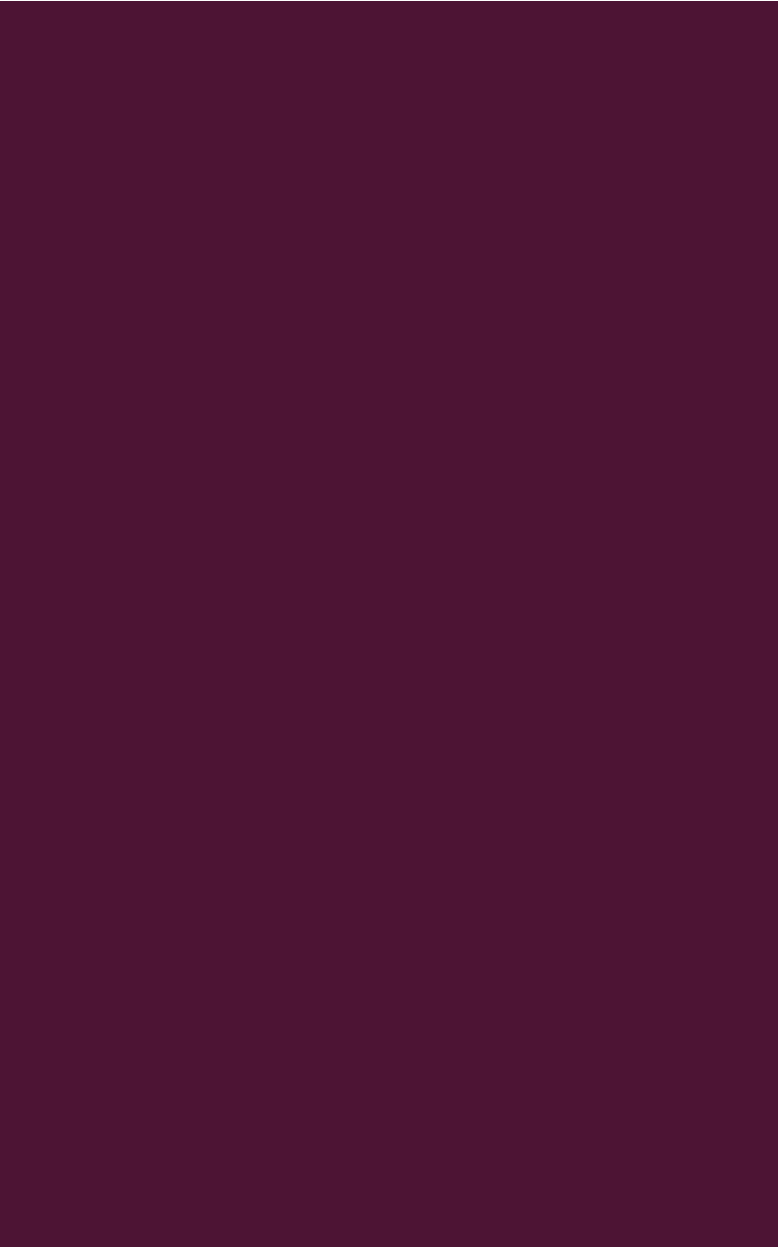
- Now, each animal can have its own `sound()`
- **Common code is written once** (`sleep()`)
- **Different behavior is enforced** (`sound()`)
- If someone forgets to define `sound()`, Java will give a **compile-time error**

```
abstract class Animal {  
    void sleep() {  
        System.out.println("Animal sleeps");  
    }  
}
```

```
    abstract void sound(); // Must be defined by  
every animal  
}
```

```
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}
```



THINK AND TELL
ANY TWO SIMILAR
SCENERIOS

ABSTRACT CLASS

- A generalized class shared by all subclasses
- Subclasses **fill in the details** according to their own implementation (overriding)
- Declared using the abstract keyword
- If a class is extending an Abstract class:
 - It must provide implementation of all of its abstract methods
- Otherwise declare that class as an Abstract class

ABSTRACT METHOD

Any method with abstract keyword

It has no body in parent class, where it is defined

Subclass must override its implementation

Abstract methods can reside only in abstract class

ABSTRACT METHOD

In regular superclasses, you often **cannot provide meaningful implementations** for every method,

Because some behaviors vary depending on the subclass.

Abstract classes solve this problem by:

- Containing **only universal/basic features** in the superclass
- Providing **prototypes (abstract methods)** for behaviors that must be implemented differently in each subclass

Subclasses then **fill in the details** by overriding the abstract methods.

RULES

- Any abstract method must belong to an abstract class.
- You cannot create an object of an abstract class (but the class can be used as a reference, as seen in polymorphism).
- Abstract methods cannot have a body.
- A child class must implement all abstract methods, or it must also be declared abstract.
- An abstract class can have both abstract and non-abstract methods.

**DID IT ONLY
SOLVE THE
PROBLEM OR
DID SOMETHING
ELSE AS WELL?**



ABSTRACTION CORE CONCEPT

- The parent class **knows every animal makes a sound**
- But it **does not know exactly how** each animal makes its sound
- So it defines sound() as **abstract**
- It **says a sound exists**, but **does not provide the implementation**
- Each child class provides the **actual implementation** of sound()
- Key Idea:** Abstraction shows “**what happens**” but hides “**how it happens**”

FULL EXAMPLE

- Animal is here abstract or superclass
- Has universal sleep method implementation
- Has prototype (abstract) sound method
- Two subclasses inherit both (abstract and normal) methods.
- Subclasses override the abstract methods for specific implementation.

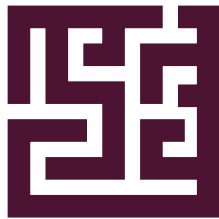
```
// Abstract Parent Class
abstract class Animal {
    // Common method for all animals
    void sleep() {
        System.out.println("Animal sleeps");
    }
    // Abstract method – every child must implement
    abstract void sound();
}

// Child class Dog
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

// Child class Cat
class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}

// Main Class
public class Main {
    public static void main(String[] args) {
        // Reference of abstract class
        Animal a1 = new Dog();
        Animal a2 = new Cat();
        // Common method
        a1.sleep();
        a2.sleep();
        // Abstract method implemented differently by each child
        a1.sound();
        a2.sound();
    }
}
```

WHAT THIS SHOWS



Problem-solving: Common behavior
(sleep()) is written once → avoids
repetition



Data hiding: Abstract method
sound() hides how the sound is
produced → enforced in each child



Abstraction essence: “Defines **what happens**, hides **how it happens**”

The background of the slide is a complex, abstract pattern. It features concentric circles of small, multi-colored squares (in shades of blue, green, yellow, red, and purple) that create a sense of depth and movement. A solid purple rectangular box is positioned on the left side of the image, containing the title text in white.

ABSTRACTION APPLICATION TO DIFFERENT FIELDS

EXAMPLE 01 – BANKING ATM

- The ATM knows you can withdraw money
- It does not show the internal banking operations
- Abstract idea: “You can withdraw money”
- Implementation: handled by the bank system internally

```
// Abstract Parent Class
abstract class ATM {
    void cardInserted() {
        System.out.println("Card inserted");
    }

    abstract void withdrawMoney(); // Abstract: implementation differs per bank
}

// Child class BankA
class BankA extends ATM {
    void withdrawMoney() {
        System.out.println("BankA: Money withdrawn using BankA process");
    }
}

// Child class BankB
class BankB extends ATM {
    void withdrawMoney() {
        System.out.println("BankB: Money withdrawn using BankB process");
    }
}

// Main
public class ATMMain {
    public static void main(String[] args) {
        ATM a1 = new BankA();
        ATM a2 = new BankB();

        a1.cardInserted();
        a1.withdrawMoney();

        a2.cardInserted();
        a2.withdrawMoney();
    }
}
```

EXAMPLE 02 – DRIVING A CAR

- You know pressing the accelerator moves the car
- You don't see the exact engine or transmission mechanism
- Abstract idea: "Press accelerator → car moves"
- Implementation: engine mechanics hidden

```
abstract class Car {  
    void start() {  
        System.out.println("Car started");  
    }  
  
    abstract void accelerate(); // Abstract: differs for each car type  
}  
  
class SportsCar extends Car {  
    void accelerate() {  
        System.out.println("SportsCar accelerates very fast");  
    }  
}  
  
class FamilyCar extends Car {  
    void accelerate() {  
        System.out.println("FamilyCar accelerates smoothly");  
    }  
}  
  
public class CarMain {  
    public static void main(String[] args) {  
        Car c1 = new SportsCar();  
        Car c2 = new FamilyCar();  
  
        c1.start();  
        c1.accelerate();  
  
        c2.start();  
        c2.accelerate();  
    }  
}
```

EXAMPLE 03 – REMOTE CONTROL

- You know pressing the volume button increases the volume
- You don't see how the signal reaches the TV or how volume circuitry works
- Abstract idea: "Volume can be increased"
- Implementation: hidden inside electronics

```
abstract class Remote {  
    void powerOn() {  
        System.out.println("TV powered on");  
    }  
  
    abstract void changeVolume(); // Abstract: implementation depends on TV brand  
}  
  
class SamsungRemote extends Remote {  
    void changeVolume() {  
        System.out.println("Samsung TV volume changed");  
    }  
}  
  
class LGRemote extends Remote {  
    void changeVolume() {  
        System.out.println("LG TV volume changed");  
    }  
}  
  
public class RemoteMain {  
    public static void main(String[] args) {  
        Remote r1 = new SamsungRemote();  
        Remote r2 = new LGRemote();  
  
        r1.powerOn();  
        r1.changeVolume();  
  
        r2.powerOn();  
        r2.changeVolume();  
    }  
}
```

EXAMPLE 04 – COOKING APPLIANCE (E.G., MICROWAVE)

- You know pressing start heats the food
- You don't see the magnetron heating mechanism
- Abstract idea: "Start → Food heats"
- Implementation: hidden inside the device

```
abstract class Microwave {  
    void plugIn() {  
        System.out.println("Microwave plugged in");  
    }  
  
    abstract void startHeating(); // Abstract: differs per brand  
}  
  
class SamsungMicrowave extends Microwave {  
    void startHeating() {  
        System.out.println("Samsung Microwave heating food");  
    }  
}  
  
class LGMicrowave extends Microwave {  
    void startHeating() {  
        System.out.println("LG Microwave heating food");  
    }  
}  
  
public class MicrowaveMain {  
    public static void main(String[] args) {  
        Microwave m1 = new SamsungMicrowave();  
        Microwave m2 = new LGMicrowave();  
  
        m1.plugIn();  
        m1.startHeating();  
  
        m2.plugIn();  
        m2.startHeating();  
    }  
}
```

KEY POINTS ACROSS ALL EXAMPLES



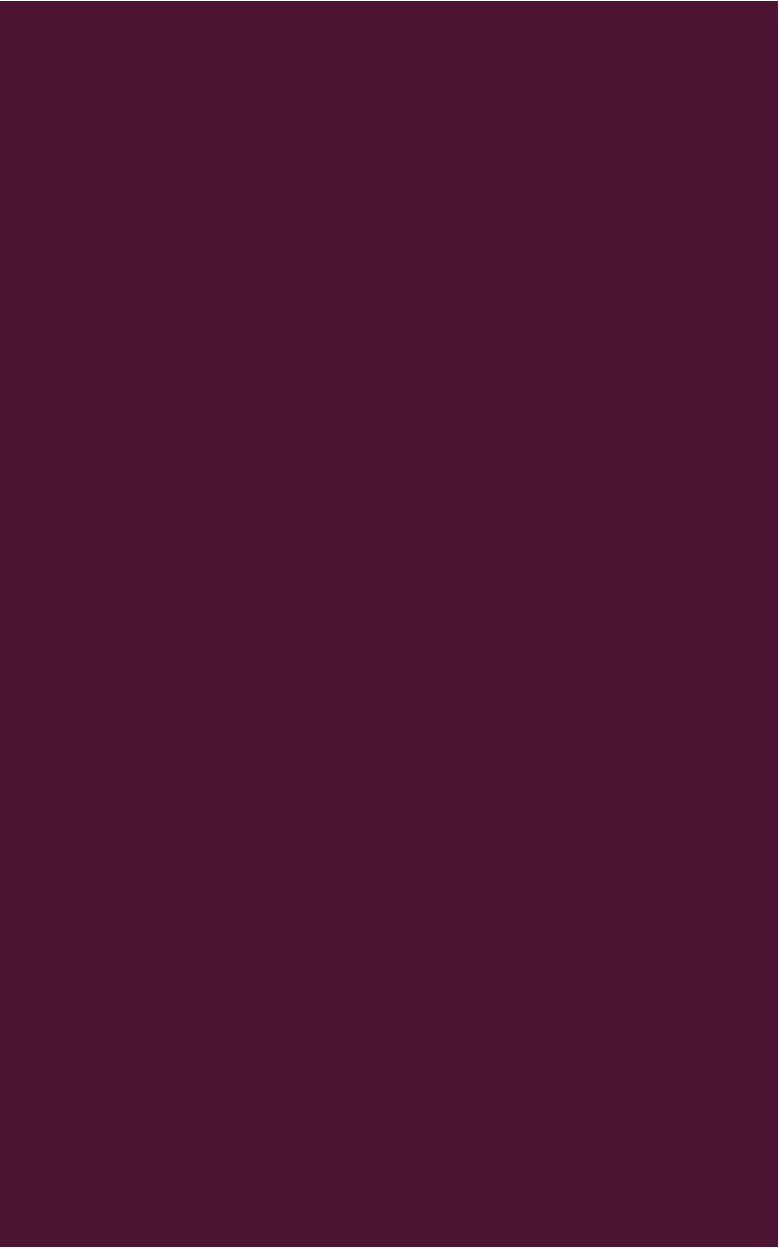
Common methods like `plugIn()`, `start()`, `sleep()`, or `powerOn()` are implemented in the abstract class → **written once**



Specific behavior (`withdrawMoney()`, `accelerate()`, `changeVolume()`, `startHeating()`) is abstract → **must be implemented by child**



Abstraction shows **what happens**, hides **how it happens**



ABSTRACT CLASSES & METHODS: LIMITATIONS

CAN AN ABSTRACT METHOD OR CLASS BE STATIC?

✗ Abstract methods cannot be static

- static methods belong to the class, not the instance
- Abstract methods must be **overridden by subclasses** → static prevents overriding

✓ Abstract classes can have static methods

- But these static methods **cannot be abstract**

CAN WE HAVE ABSTRACT CONSTRUCTORS?

✗ Abstract constructors are not allowed

- Constructors are used to **initialize objects**
- Abstract classes **cannot be instantiated**, so abstract constructors don't make sense

✓ Abstract classes can have **regular constructors**

- Called **when a subclass object is created**

CAN ABSTRACT CLASSES OR METHODS BE FINAL?

1. Abstract Classes:

- ✗ Cannot be final
- final means **cannot be inherited**
- Abstract classes are meant to be **extended**
- Combining abstract + final is **contradictory**

2. Abstract Methods:

- ✗ Cannot be final
- final methods **cannot be overridden**
- Abstract methods **must be overridden** in subclasses
- Combining abstract + final is **contradictory**

CAN VARIABLES BE ABSTRACT IN JAVA?

- **✗ No, variables cannot be declared as abstract**
 - Abstract means **no implementation provided**
 - Variables **must have a type and storage**, so they always exist in memory
 - Only **methods** can be abstract because their **implementation can be deferred to subclasses**